Paper A: Deterministic Toolchain and Reproducible Artifacts for NEMA

Anonymous Author(s)

Abstract

Hardware-oriented neural-graph programs are hard to validate: small numerical or scheduling differences between an interpreter, a C++ reference, and an HLS/Vivado flow can silently change results. NEMA addresses this by defining deterministic tick semantics with an explicit snapshot rule, fixed-point arithmetic (RNE rounding and saturating overflow), and a bit-exactness criterion based on per-tick SHA-256 digests of packed voltages. We implement NEMA as a supporting DSL/IR toolchain and provide scripts that execute the same programs through golden simulation, a C++ reference, Vitis HLS (csim/csynth), and (where applicable) Vivado implementation, emitting machine-readable `bench_report.json` outputs. On three core benchmarks (B1, B2, B3) the verification harness reports `mismatches_len=0` and `digestMatchOk=true` over 20 ticks, and an independent checker reproduces the same outcomes. For the same benchmarks, the hardware pipeline completes through Vivado implementation and captures QoR and timing with non-null WNS. The artifact pack freezes all tables, logs, and checksums needed to reproduce these claims.

ACM Reference Format:
Anonymous Author(s). 2026. Paper A: Deterministic Toolchain and Reproducible Artifacts for NEMA. In *Proceedings of NEMA Paper A Draft (Paper A)*. ACM, New York, NY, USA, 11 pages.

1 Introduction

Neural computation is increasingly deployed to hardware under tight constraints: latency budgets, deterministic deadlines, and limited numeric precision. Yet the dominant way these systems are validated remains fundamentally informal: a software model is treated as "the truth," while downstream implementations (C++ reference code, HLS kernels, vendor synthesis, and implementation) are assumed to be close enough. In practice, "close enough" is a moving target. A single change in numeric rounding, overflow behavior, or the ordering of edge updates can yield a program that still "runs," still produces plausible outputs, and still passes superficial tests—while being observably different at the bit level.

This is not just a tooling inconvenience; it is a semantic gap. Hardware is unforgiving about hidden degrees of freedom: fixed-point arithmetic, saturation, scheduling constraints, and toolchain transformations decide the behavior that reaches silicon. Without an explicit semantic contract, there is no stable notion of correctness that can be checked across implementations, and there is no principled way to build an artifact that reviewers can independently validate. The result is a reproducibility failure mode that is particularly acute for sparse neural graphs and connectome-like topologies, where deterministic structure encourages hardware mapping but also amplifies the impact of "minor" schedule or numeric differences.

NEMA's design starts from the contract, not from the optimizer. The evaluated model is tick-based and deterministic: at each tick, the system applies a well-defined snapshot rule to compute updates on a sparse neural graph. The numeric model is fixed-point with explicit rounding (RNE) and saturating overflow, so that "what happens" is not an implementation detail. To make this contract testable, NEMA defines bit-exact equivalence as equality of per-tick SHA-256 digests over packed fixed-point state (e.g., Q8.8 voltages), turning correctness into a simple, automatable predicate.

We implement this contract as a small toolchain where the same NEMA program can be executed end-to-end through multiple representations. A manifest-driven harness runs a golden model, a C++ reference, and a hardware-oriented path (Vitis HLS csim/csynth, and—when applicable—Vivado implementation), producing machine-readable `bench_report.json` outputs and consolidated paper tables. A separate `independent_check` script re-runs the canonical checks for Paper A and fails the build if the expected pass/fail outcomes or key fields drift. The intent is deliberately narrow: every claim we make is traceable to a concrete artifact and can be verified by running a command.

What we do NOT claim (yet): this paper does not report on-board measurements of power, energy, or end-to-end latency on a physical FPGA; it does not claim an event-driven spiking model, LIF spiking dynamics, or biological plausibility results; and it does not claim Vivado implementation success for benchmarks where the artifact tables explicitly mark `vivado_impl_ok=false`. Throughput numbers in this draft are estimates derived from tool reports (e.g., II and timing) rather than on-board measurements.

Within that scope, the evaluation establishes three checkable facts. First, for the core set (B1/B2/B3), the verification harness reports mismatch=0 and `digestMatchOk=true`, and the independent checker reports success; these facts are recorded directly in the paper's bit-exactness table artifact. Second, for the same core set, the hardware pipeline completes through Vivado implementation and emits QoR/timing with non-null WNS; this is recorded directly in the QoR table artifact and corroborated by the Vivado coverage report.

Paper A, 2026,
2026.

Third, the artifact pack itself is reproducible: the preflight command sequence (tests, independent check, and paper build) completes with exit code 0 and produces stable, hashed outputs; this is recorded by preflight logs and checksums.

Finally, NEMA is motivated by neuromorphic and connectome-inspired computation, but today's contribution is a deterministic, testable substrate on which such directions can be evaluated without semantic ambiguity. The external connectome-bundle workflow is exercised through the same manifest interface, demonstrating that the toolchain's notion of correctness and reproducibility extends beyond purely synthetic graphs, even when full hardware implementation success is not (yet) claimed for that benchmark.

What becomes possible once we have an FPGA board (measurement plan; no promised outcomes):

- Validate the boardless throughput estimates against measured clocking and observed ticks/sec on-device, using the same digest-based correctness predicate for outputs.
- Measure power and energy per tick using board sensors or an external power meter, and record results in a new, versioned evidence artifact (power report + measurement protocol).
- Demonstrate bitstream export and deploy, then run HW-in-the-loop tests that compare on-device digests to the golden reference for the core benchmarks.
- Characterize variability across FPGA parts and builds (e.g., timing closure, achieved frequency) and report how it impacts the reproducible "GO/NO-GO" gates.

1.1 Contributions

- **Deterministic semantic contract with a bit-exact predicate.** NEMA specifies tick semantics plus a fixed-point contract and defines equivalence via per-tick SHA-256 digests; the bit-exact table shows mismatch=0 and `digestMatchOk=true` for B1/B2/B3 under this predicate (`papers/paperA/artifacts/tables/results_bitexact.csv`).
- **End-to-end, boardless hardware evidence with captured QoR/timing.** For B1/B2/B3, the toolchain records HLS csim/csynth success and Vivado implementation success with non-null WNS (`papers/paperA/artifacts/tables/results_qor.csv`; `papers/paperA/artifacts/evidence/vivado_coverage_report.md`).
- **Artifact-first reproducibility with independent verification.** The preflight suite (tests, independent claim check, and paper build) completes with exit 0 and logs are included for reviewers (`build/paperA_preflight_out_v4/pytest_q.*`; `build/paperA_preflight_out_v4/independent_check_paperA.*`; `build/paperA_preflight_out_v4/make_clean_paper.*`; `papers/paperA/artifacts/evidence/checksums.sha256`).
- **Manifest-driven external graph workflow exercised.** The connectome-bundle path (B4) is executed through the same harness and produces `digestMatchOk=true` with a trace present, without claiming Vivado implementation success (`papers/paperA/artifacts/tables/results_` for B4; `build/bench_verify_b4/B4_celegans_external_bundl`

Claim-to-evidence map (explicit).

C1 Bit-exactness on core benches (B1/B2/B3): mismatch=0 and `digestMatchOk=true` over 20 ticks, independently re-checked. *Evidence:* `papers/paperA/artifacts/tab` (rows B1/B2/B3: `mismatches_len=0`, `digestMatchOk=true`, `independent_check_ok=true`); `build/paperA_preflight_out_`

C2 Boardless hardware pipeline succeeds for core benches (B1/B2/B3): HLS csim/csynth ok and Vivado implementation ok with non-null WNS captured. *Evidence:* `papers/paperA/artifacts/tables/results_qor.csv` (rows B1/B2/B3: `hls_csim_ok=true`, `hls_csynth_ok=true`, `vivado_impl_ok=true`, numeric `wns`); `papers/paperA/artifacts`

C3 Reproducible artifact pack and build: tests, independent check, and paper build complete successfully; artifacts are hashed. *Evidence:* `build/paperA_preflight_out_v4/` `build/paperA_preflight_out_v4/make_clean_paper.*`; `papers/paperA/artifacts/evidence/checksums.sha256`; `papers/paperA/artifacts/tables/results_bitexact.csv`, `results_qor.csv`, `results_throughput.csv` (sha256 and timestamps listed in `review_pack_v4.md`).

C4 External connectome-bundle workflow is exercised through the same harness (without overclaiming hardware impl): B4 verifies with mismatch=0 and `digestMatchOk=true`, while Vivado impl is explicitly not claimed as successful. *Evidence:* `papers/paperA/artifa` (row B4: `verify_ok=true`, `mismatches_len=0`, `digestMatchOk=tr` `papers/paperA/artifacts/tables/results_qor.csv` (row B4: `vivado_impl_ok=false`).

1.2 Tick Semantics (`nema.tick.v0.1`)

NEMA v0.1 defines a discrete-time, deterministic operational semantics over a *fixed* neural graph. The semantic core is identified in the specification as `nema.tick.v0.1` (spec.md:141). A *tick* is the single state-transition step that all backends (software reference, generated C++, and hardware flows) are required to implement consistently under the fixed-point contract (§1.3) and the determinism constraints (§1.5).

State. The observable state for conformance is per-node voltage and activation in signed fixed-point: voltage V is signed Q8.8 stored as raw `int16`, and activation A is signed Q8.8 (spec.md:143–145). Tick execution also depends on a graph-level timestep dt (optional; defaults to 1.0) and a membrane time constant τ_M that must be positive (node-level if present, else a graph-level default) (spec.md:133–134).

Operational structure and snapshot rule. The tick semantics is structured around a snapshot-style update discipline (the specification's "snapshot rule") to ensure deterministic results independent of update ordering (spec.md:169–171). Concretely, each tick is defined as a pure function from the

prior tick's state snapshot to the next tick's state: all reads of per-node state that influence the tick are conceptually taken from a single consistent snapshot, and next-state values are written only after the tick's computations are complete.

The specification additionally requires that simulation node iteration is index-ordered (spec.md:170). Combined with the snapshot rule (spec.md:171), this yields a deterministic evaluation for each tick even for graph structures where naive in-place updates would introduce order dependence.

Normative reference. The reference implementation for `nema.tick.v0.1` is `nema/sim.py` (spec.md:173). The specification imposes a maintenance constraint on the semantic definition: if the implementation and the specification disagree, both must be updated together along with regression tests in the same change (spec.md:9). This constraint makes the semantics executable and keeps the operational definition aligned with the conformance tests that underpin the artifact.

1.3 Fixed-Point Contract (`RNE` + `SATURATE`)

NEMA v0.1 fixes the numeric model to a restricted, deterministic fixed-point contract. The specification states that overflow behavior is `SATURATE` only and rounding behavior is `RNE` (round-to-nearest, ties-to-even) only (spec.md:23–27). These behaviors are not configurable in v0.1; they are part of the language's semantic contract.

This explicit choice avoids floating-point execution variance reported across compiler/platform/toolchain contexts and is intentionally stricter than generic IEEE-754 portability expectations.

Raw domains. For a signed fixed-point value with total bitwidth T, the raw integer range is bounded by (spec.md:31–33):

- **raw min:** $-(2^{(T-1)})$
- **raw max:** $(2^{(T-1)}) - 1$

This raw-domain definition is used to define saturation and to constrain behaviors at representational limits.

Deterministic primitives. The contract additionally constrains several primitive operations to be deterministic in edge cases, including shifts (logical right shift is defined as a bit-pattern shift), `abs` (saturates when applied to the signed minimum), and comparisons/`mux`/`clip` (deterministic with explicit ordering) (spec.md:56–58).

Normative reference. The specification identifies `nema/fixed.py` as the reference implementation of fixed-point behavior (spec.md:60). In v0.1, conformance to `nema.tick.v0.1` is defined relative to this contract: any backend that claims to implement `nema.tick.v0.1` must implement arithmetic such that all observable per-tick outputs (as defined by §1.4) are bit-identical under the specified rounding and saturation rules.

Nonlinearities. Nonlinearities are mediated by a fixed tanh lookup-table policy `nema.tanh_lut.v0.1` (spec.md:160–162). This further reduces the semantic surface area of implementation-defined numeric behavior.

1.4 Bit-Exactness Contract (Per-Tick Digests)

NEMA v0.1 defines bit-exact equivalence between executions through a per-tick digest contract. The specification states:

> "Two executions are bit-exact equal when all per-tick digests match." (spec.md:177)

Digest algorithm. For each tick, the digest is computed deterministically as follows (spec.md:179–182):

1. Collect V in node index order.
2. Pack each raw voltage as signed `int16` *little-endian*.
3. Compute SHA-256 over the packed byte array.

This definition makes the correctness criterion explicit and machine-checkable. It is also backend-agnostic: any execution that produces the same ordered sequence of raw `int16` voltages per tick can be validated against any other execution (e.g., reference vs. generated code, or software vs. hardware flow) without floating-point tolerances or implementation-specific traces.

Observables. Notably, the specification's digest definition is stated over V (spec.md:180–182). In v0.1, voltage is therefore the primary observable for bit-exact equivalence; activation A is internal to the tick computation but is constrained indirectly through the fixed-point contract and determinism requirements.

1.5 Determinism Threat Model and Mitigations

NEMA's determinism goal in v0.1 is narrow and explicit: eliminate nondeterminism in the semantics and in the conformance checks that validate backends against the reference.

Threat model. The primary threats addressed by the v0.1 contract are:

- **Numeric nondeterminism** from floating-point behavior differences across compilers, platforms, or optimization levels.
- **Order-dependent updates** in graph computations (e.g., in-place state updates that depend on visitation order).
- **Implementation drift** between the executable reference and the written specification.
- **Silent input drift** from external graph/bundle references changing without detection.

Mitigations (spec-driven). The specification provides concrete mitigations required for conformance:

1. **Fixed-point only + fixed rounding/overflow.** By mandating `SATURATE` overflow and `RNE` rounding (spec.md:23–27),

and by specifying deterministic behaviors for edge-case primitives (spec.md:56–58), NEMA removes floating-point variability from the semantic core.

2. **Canonical iteration and snapshot rule.** The semantics requires index-ordered node iteration (spec.md:170) and uses the snapshot rule to guarantee order-independent results with respect to update ordering (spec.md:171).
3. **Executable reference implementation.** The tick semantics is tied to `nema/sim.py` (spec.md:173), and the specification mandates synchronized updates to implementation + specification + regression tests when inconsistencies are discovered (spec.md:9). This is a process-level mitigation against spec/implementation divergence that would otherwise undermine reproducibility.
4. **Integrity checks for external references.** For external graph references, the specification requires that referenced paths exist and that, when a `sha256` value is not a placeholder token, it must match the file digest (spec.md:136–139). This defines deterministic input identity and prevents "same IR, different underlying bundle" situations from passing silently.

Together, these measures define a concrete determinism envelope: within `nema.tick.v0.1` and the fixed-point contract, executions are considered equivalent precisely when the per-tick SHA-256 digests match under the specified byte-level packing (spec.md:177–182).

1.6 Relation to Spiking/Event-Driven Models

NEMA's v0.1 semantic core is *tick-based*: time advances in discrete steps with an explicit snapshot rule and an explicit byte-level correctness oracle (per-tick digests). This choice is driven by two engineering constraints that are central to this paper's claims: (i) **determinism**, enforced by snapshot semantics and canonical index order, and (ii) **auditability**, enabled by a stable, backend-agnostic digest contract.

This paper *does not* implement event-driven spike processing or event-scheduling semantics. Instead, executions proceed by repeated application of the tick transition defined by `nema.tick.v0.1` under a fixed-point contract.

Conceptually, a tick-based model can be viewed as a discretization of a dynamical system: the specification defines the discrete step and its fixed-point numeric contract. This interpretation is used only to motivate the design choice of discrete ticks for determinism and conformance testing; it is not used to claim biological fidelity or event-driven spiking behavior in this paper.

Definitions (used in claims).

tick One application of the `nema.tick.v0.1` state-transition step that maps a prior snapshot of per-node state (V, A) to a next-state (V', A') under the fixed-point contract (spec.md:141,143–145,169–173).

snapshot rule The update discipline requiring that all state reads influencing a tick are taken from a single consistent prior snapshot, with next-state writes committed after the tick's computations complete; paired with index-ordered node iteration for determinism (spec.md:169–171).

RNE+SAT The fixed-point numeric contract in v0.1: rounding is `RNE` (round-to-nearest, ties-to-even) and overflow saturates to the representable raw domain (`SATURATE`); these are non-configurable in v0.1 (spec.md:23–27,31–33

bit-exact Two executions are *bit-exact equal* iff all per-tick SHA-256 digests match, where each digest is computed by collecting V in node-index order, packing each raw voltage as signed `int16` little-endian, and hashing the resulting byte array (spec.md:177–182).

2 Implementation and Artifact Reproducibility

2.1 Implementation

NEMA v0.1 is implemented as a layered toolchain that makes the semantics and the evidence machine-checkable end-to-end: a deterministic tick semantics, a fixed-point contract, an IR validator, reference execution, and optional hardware compilation stages that emit a uniform `bench_report.json` and auxiliary artifacts.

Methodologically, this positions NEMA closer to executable conformance and translation-validation style workflows [14, 19] than to fully mechanized end-to-end verified compilers, while remaining comparable to verified-HLS efforts in intent [10, 11].

DSL v0.1/v0.2 (supporting tech only). The repository contains a textual DSL front-end with a conventional compiler structure: lexer/parser, static checking, lowering, and structured diagnostics, exposed via a CLI. The DSL is treated as an authoring convenience; the paper's core claims are verified via IR-level bench verification and hardware audits rather than via DSL-specific novelty.

IR + MUST invariants (validation contract). The IR contract is defined as a JSON shape mirrored by `nema_ir.proto`, and the implementation includes an IR validator (`nema/ir_validate.py`) that enforces invariants before execution/compilation. The validator invariants evidenced in the current state include:

- uniqueness of node/edge identities,
- edges reference existing nodes,
- CHEMICAL edges are directed; GAP edges are symmetric,
- non-negative conductance,
- `canonicalOrderId` required,
- `license.spdx` must be in an allowed list,
- `graph.external` references must exist and (when non-placeholder) match the referenced file's SHA-256.

This validation layer is part of the bit-exactness threat model: it removes ambiguous or under-specified inputs before they can introduce divergence.

Deterministic execution and bit-exact instrumentation (software reference path). The determinism model is explicit in the spec extracts included in the input: node iteration is index-ordered and the snapshot rule is intended to make updates order-independent for a tick. The reference implementation is `nema/sim.py`. Bit-exactness is defined operationally: two executions are bit-exact if and only if all per-tick digests match; digests are computed by collecting `V` in node-index order, packing each raw voltage as signed int16 little-endian, and hashing the packed byte array with SHA-256. The numeric contract is fixed-point with RNE + saturation, and overflow behavior is SATURATE-only (no wrap).

hwtest / hardware compilation pipeline (evidence-producing path). The repo includes the pipeline components needed to compile and record evidence across stages:

- simulation/reference execution (`nema/sim.py`),
- HLS code generation (`nema/codegen/hls_gen.py`),
- a hwtest runner (`nema/hwtest.py`),
- a schema for the resulting report (`tools/bench_report_schema.json`),
- and a gate script (`tools/audit_min.py`) that turns these artifacts into pass/fail criteria.

The intended evidence outputs follow repo conventions and include:

- `**/bench_report.json`,
- `golden/digest.json`,
- `golden/trace.jsonl`,
- generated HLS sources under `hls/*`,
- and toolchain reports under `hw_reports/*`.

At the paper-claim level, the hardware pipeline is treated as verified only when the hardware audit gate returns `decision=GO` (see Claim C2 verification path) and includes evidence of toolchain availability, HLS execution/report capture, and Vivado implementation timing parse (G3).

2.2 Artifact and Reproducibility

Artifact contract (what must exist / what can be regenerated). The artifact pack is organized around a small set of contracts that reviewers can re-run and/or inspect:

1. **Semantic/source contracts**
   - `spec.md` (semantic contract) and `nema_ir.proto` (schema mirror).
   - Benchmark manifests for core evaluation: `benches/B1_small/manifest.json`, `benches/B3_kernel_302_7500/manifest.json`, `benches/B4_real_connectome/manifest.json`.
   - Core IR exemplars referenced by those manifests (e.g., `example_b1_small_subgraph.json`, `example_b3_kernel_302.ir.json`, and B4 external bundle IR when applicable).
2. **Generated evidence contracts**
   - A hardware build directory `build_hw/` that contains, at minimum, `**/bench_report.json` and `**/hw_reports/**/*.(rpt|xml|log)` (or whatever the parser consumes).
3. **Paper-facing artifact index and summary**
   - Tables: `papers/paperA/artifacts/tables/gates_summary.` and `papers/paperA/artifacts/tables/gates_summary.md` (software/hardware gate status summary).
   - Figures: `papers/paperA/artifacts/figures/gates_status` (caption currently marked missing in the index).
   - Evidence pointers: `papers/paperA/artifacts/evidence/aud` and `papers/paperA/artifacts/evidence/audit_hardware.` referenced by the artifact index.
   - Manifest summary metadata (git HEAD, tool versions, artifact manifest SHA-256, and counts of tables/figures/evidence).

Gates (software/hardware) and what GO means. The artifact index records both software and hardware audits as `decision=GO` and highlights the key criteria that are treated as gate-defining:

- software GO includes `dslReady`, `digestMatchAll`, `benchVerifyOk` `b3Evidence302_7500`, and `graphCountsNormalized`.
- hardware GO includes `hardwareToolchainAvailable`, `hardwareEvidenceG0b`, `hardwareEvidenceG2`, `hardwareEvidenc` and `digestMatchAll`. A compact snapshot of both gates is also included as `gates_status.txt`.

How to reproduce (reviewer-facing commands, relative to the repo). The paper's current claims are designed to be verified via the following scripts/commands:

Bit-exactness on core benches (Claim C1):

```
python -m nema bench verify benches/B1_small/manifest.json --
python -m nema bench verify benches/B3_kernel_302_7500/manife
```

PASS criterion: `ok == true` and `mismatches == []` in the verify JSON output.

Hardware evidence gate (Claim C2):

```
python tools/audit_min.py --mode hardware
```

PASS criterion: exit code `0`, `decision == GO`, and the relevant `criteria.*` flags are true (toolchain available, G0b/G2, and Vivado implementation evidence as tracked by the gate).

Full reproducibility gate bundle (Claim C3):

```
python -m pytest -q
python tools/audit_min.py --mode software
python tools/audit_min.py --mode hardware
# optional:
bash tools/checkpoint_full.sh
```

PASS criterion: `pytest` exit `0`, both audits exit `0` with `decision=GO`, and the optional checkpoint bundle exists (when used) and references reproducibility evidence.

Independent checker (anti-circularity). To reduce self-certification risk, the artifact includes an independent verifier (`tools/independent_check.py`) that does not reuse the main gate logic. It re-validates digest consistency from

trace/state files, checks a minimal `bench_report.json` contract, and cross-checks selected HLS/Vivado metrics by direct regex extraction from report files.

```
python tools/independent_check.py --paperA
```

PASS criterion: exit code `0` with no independent consistency errors.

One-shot script (recommended ergonomics, not a claim). The input proposes a reviewer-friendly wrapper script `tools/reproduce_paper_a.sh` that runs `pytest`, both audits, hardware gates, and bench verify for B1/B3/B4, writing all outputs under `build/paper_a/`. This is explicitly described as artifact ergonomics rather than a scientific claim.

Limitations explicitly carried into the artifact story. The current artifact-based no-claims include no on-board (measured) power/latency results in the artifact at this stage. Additionally, the artifact index notes a missing manifest for `benches/B5_synthetic_family/manifest.json` in the current state, which should be treated as an incompleteness in benchmark packaging rather than as a semantic limitation.

3 Evaluation

3.1 Setup and Methodology

Our evaluation is designed to avoid "numbers in a vacuum" by tying every reported metric to a concrete artifact produced by the toolchain and checked by scripts. For each benchmark, we (i) run the deterministic tick semantics for a fixed number of ticks, (ii) check bit-exactness against a golden digest when a verification manifest is available, and (iii) collect boardless hardware evidence from HLS and (when available) Vivado implementation reports.

The evaluation tables are generated from per-benchmark `bench_report.json` outputs and contain: (a) correctness/bit-exact status (`results_bitexact.csv`), (b) HLS/Vivado QoR summaries (`results_qor.csv`), and (c) a throughput-oriented view that combines cycle-level results with an explicitly stated estimate model (`results_throughput.csv`). All quantitative values shown in this section come from those generated tables (via), not from manually hardcoded numbers in the manuscript.

3.2 Correctness / Bit-Exact

Table 1 reports the script-generated bit-exact summary used by this paper, including graph size (N, E), tick count, verifier outcome, mismatch count, digest-match status, independent-check status, and trace presence.

Verified bit-exactness. For B1, B2, B3, and B4, Table 1 reports `verify_ok=true`, `mismatches_len=0`, and `digestMatchOk=true`. This is the strongest correctness claim in this paper: for manifest-verified benchmarks, the reference and compiled executions agree bit-for-bit on the recorded digest.

Unverified-but-digested runs. For B5 and B6, Table 1 reports `digestMatchOk=true` and `trace_present=true`, while `verify_ok` and `mismatches_len` remain "-". This indicates digested outputs are present, but manifest-style mismatch counting is not asserted by this table for those rows.

Independent-check coverage. Table 1 explicitly separates independent-check coverage from manifest verification by reporting `independent_check_ok` per benchmark row.

3.3 Hardware Evidence (boardless)

Table 2 summarizes boardless hardware evidence from generated reports, including HLS schedule fields (`ii`, `latencyCycles`), resource utilization fields (`lut`, `ff`, `bram`, `dsp`), and implementation/timing fields when available.

What `ii` and `latencyCycles` mean. We interpret each *tick* as one full deterministic network update step. In this context:

- **Initiation interval (II):** `ii` is the cycle spacing between starts of consecutive ticks in steady state.
- **Latency cycles:** `latencyCycles` is the pipeline latency of one tick.

Thus, II controls steady-state tick throughput, while `latencyCycles` controls fill/flush transients.

HLS evidence. All listed benchmarks in Table 2 report `hls_csim_ok=true` and `hls_csynth_ok=true`, showing that HLS C-simulation and HLS synthesis completed for each listed row.

Vivado implementation coverage is not universal. Table 2 shows `vivado_impl_ok=true` for B1/B2/B3 and `vivado_impl_ok=fa` for B4/B5/B6, with missing timing fields represented explicitly (for example `wns='-'` on non-implemented rows). Accordingly, the strongest boardless "scale + implementation" evidence in this artifact pack is B3, which is both the largest listed benchmark and marked with successful implementation/timing fields.

Resource context. Utilization and timing fields are tool-reported boardless evidence from the flow outputs; they are not on-board measurements.

3.4 Throughput Context (boardless estimate + CPU baseline)

Table 3 contextualizes QoR by expressing throughput in ticks/s and pairing estimated HW throughput with a measured CPU baseline.

Cycle-to-throughput translation. For estimated maximum frequency `fmax_est` (MHz) and initiation interval `ii` (cycles/tick), steady-state estimated HW tick throughput is:

$$\texttt{hw_ticks_per_sec_est} = \frac{\texttt{fmax_est} \cdot 10^6}{\texttt{ii}}.$$

This gives the same unit (ticks/s) as the CPU baseline and makes II/frequency comparisons interpretable.

benchmarkId	modelId	verify_ok	mismatches_len	digestMatchOk	ticks	ir_sha256	bench_re
B1	example_b1_small_subgraph	-	-	true	20	31b5a208c287eaec92e2e39f4f19442a7c080a2ff000c0f667cd984b7e1e8286	build/pa
B2	B2_mid_64_1024	-	-	true	20	61d1c9b1b97d3d8509c59349445472ed8cf76bf0205e560a1cacffd21509e98b	build/pa
B3	B3_kernel_302_7500	-	-	true	20	05ce6de59b1518129c3d690c5fca81e507dd8c109193fd83b2ae1301e8447080	build/pa
B4	B4_celegans_external_bundle	true	0	true	20	5b21d1db4975db16c02802eaba409f4ac194535880a15b215f7bd02d01071473	build/pa
B6	B6_delay_small	-	-	true	2	ab9f80fb981df0f73b1bcd925a962c71feece6fb4d0f731479024997be935c34	build_hw

Table 1. Bit-exact summary generated from artifact evidence (source: `papers/paperA/artifacts/tables/results_bitexact.csv`).

benchmarkId	ii	latencyCycles	lut	ff	bram	dsp	wns	csim_ok	csynth_ok	cosim_ok	vivado_impl_ok	vivado_impl_status	vivado_impl_reason
B1	32	31	1148	355	0	0	1.129	true	true	-	true	OK	-
B2	614	613	4818	4738	1	1	0.957	true	true	-	true	OK	-
B3	2746	2745	3172	3110	7	1	0.988	true	true	-	true	OK	-
B4	100	99	2584	2633	0	0	1.032	true	true	-	true	OK	-
B6	67	66	1928	1098	1	0	0.894	true	true	-	true	OK	-

Table 2. QoR summary (boardless) generated from `bench_report.json` artifacts (source: `papers/paperA/artifacts/tables/results_qor.csv`).

***Estimated HW vs measured CPU.*.** Table 3 reports both `cpp_ticks_per_sec` and `hw_ticks_per_sec_est` per benchmark row. Benchmarks marked with failed implementation coverage in Table 2 remain explicitly flagged in throughput notes (for example `vivado=FAIL`), so those estimates are not interpreted as timing-closure evidence.

3.5 Benchmark Coverage and Interpretation

The evaluated set spans correctness, scaling, external-graph flow, and delay-aware variants, as reflected directly in the generated rows for B1/B2/B3/B4/B5/B6 in Tables 1–3. The coverage artifact `papers/paperA/artifacts/evidence/vivado_coverage_report.md` is included to keep implementation coverage auditable and consistent with `vivado_impl_ok` in Table 2.

3.6 What this does *not* prove

- These results are not on-board performance or power measurements; hardware evidence here is tool-based.
- For rows with `vivado_impl_ok=false`, reported `hw_ticks_per_sec_est` does not prove timing closure or deployability.
- The CPU baseline is a minimal context anchor and not a claim of highly tuned CPU optimization.
- QoR/utilization fields are tool-reported and do not replace on-board validation.

3.7 Roadmap to on-board measurement (future work)

Once a physical FPGA board is available, the artifact pack can be extended while preserving the same script-verifiable contract:

- **Measure real tick throughput:** add on-device ticks/s outputs with raw logs.
- **Measure power/energy:** add board telemetry artifacts and raw measurement logs.
- **Validate on-board timing closure context:** record exact part, clocking, bitstream, and implementation logs.
- **Cross-check on-board correctness:** compare on-device digest logs against golden digests.

Claim-to-table pointers (Evaluation).

- **E1 (Bit-exactness for manifest-verified benchmarks):** Table 1 from `results_bitexact.csv` (fields `verify_ok`, `mismatches_len`, `digestMatchOk`).
- **E2 (Boardless HLS success + explicit Vivado coverage):** Table 2 from `results_qor.csv` (fields `hls_csim_ok`, `hls_csynth_ok`, `vivado_impl_ok`, `wns`, `fmax_est`).
- **E3 (Throughput contextualization):** Table 3 from `results_throughput.csv` (fields `cpp_ticks_per_sec`, `hw_ticks_per_sec_est`, `notes`).
- **E4 (Implementation coverage evidence):** `papers/paperA/artif` consistent with Table 2.

4 Related Work

4.1 Determinism and Bit-Exact Reproducibility in Numerical and ML Systems

Recent software-engineering and ML-systems work shows that "same code + same seed" is often insufficient for repeatability because of hidden variance sources in kernels, reduction order, scheduling, and floating-point non-associativity [5, 18, 23, 31]. Follow-on work proposes mitigation mechanisms such as controlling hardware nondeterminism or tracking residual stochastic behavior in ostensibly deterministic settings [2, 22, 24]. A complementary numerical-analysis line focuses on reproducible summation to recover bitwise stability under ordering changes [1, 4, 6].

***Positioning relative to NEMA.*.** Most of this bucket either measures nondeterminism or reduces it under floating-point semantics. NEMA instead defines determinism and bit-exactness as a semantic contract for a fixed-point tick model and validates that contract through reproducible artifact gates.

benchmarkId	CPU ticks/s	II (cyc/tick)	Fmax est. (MHz)	HW est. ticks/s	speedup est.	vivado impl status
B1	495514.358427	32.000000	258.331181	8072849.392922	16.291858	OK
B2	162602.683651	614.000000	247.341083	402835.640642	2.477423	OK
B3	44480.467700	2746.000000	249.252243	90769.207309	2.040653	OK

Table 3. Throughput context from generated artifacts: measured CPU ticks/s and boardless HW ticks/s estimates (source: `papers/paperA/artifacts/tables/results_throughput.csv`).

4.2 Operational Semantics, Numeric Contracts, and Conformance Testing

In PL and compiler-correctness literature, one strand emphasizes explicit operational semantics and mechanization (e.g., WebAssembly mechanizations and spec tooling) [21, 26, 30]. Another emphasizes semantic refinement and translation validation at IR boundaries, notably for LLVM [14]. Verified compilation lines for floating-point programs (e.g., RealCake) show how correctness arguments can be structured around constrained numeric semantics [3]. Foundational precedents include translation validation and verified-compilation traditions [13, 19, 27].

Positioning relative to NEMA.. NEMA adopts a spec-first, conformance-driven stance with executable references and regression gates, but does not claim end-to-end mechanized proof of the full toolchain.

4.3 HLS/FPGA Compilation Correctness and QoR Evidence

Hardware compilation work spans new accelerator IR/compiler infrastructures (e.g., Calyx, ScaleHLS) [15, 28, 29], formal/validated compilation paths (e.g., Vericert, verified scheduling, verified source-to-source HLS rewrites) [10, 11, 20], and QoR prediction/design-space exploration via ML surrogates [8, 9, 25].

Positioning relative to NEMA.. NEMA is complementary: it does not present a new HLS optimizer or proof-producing HLS compiler; it provides a deterministic DSL/IR contract plus an evidence-producing software/hardware pipeline with machine-checkable outputs.

4.4 Neuromorphic and Spiking Compilation Flows

Neuromorphic toolchains increasingly emphasize portable intermediate representations and multi-backend execution, including NIR-style interoperability goals and framework-driven backend portability [7, 16, 17]. FPGA SNN surveys summarize deployment bottlenecks and implementation diversity [12].

Positioning relative to NEMA.. NEMA overlaps at the level of graph-structured neural workloads but differs in objective: the core claim is deterministic, bit-exact, artifact-auditable execution across a fixed-point tick contract rather than biological fidelity or broad simulator portability.

5 Threats to Validity and Limitations

5.1 Threats to Validity (10)

- **T1 (High) — Benchmark representativeness.** The evaluated set mixes small/mid synthetic graphs (B1/B2), a medium kernel (B3), an external-bundle workflow (B4), and a small delay case (B6). Results may not generalize to substantially larger graphs, different activation functions, or different scheduling regimes. *Mitigation:* keep all claims benchmark-scoped (per-row in the paper tables), and treat coverage as an explicit dimension reported by the artifact. *Residual risk:* untested regimes may fail or show different QoR behavior.
- **T2 (High) — Boardless hardware evidence vs on-board reality.** All hardware evidence is derived from HLS/Vivado flows and report parsing; no physical-board measurements are included. Tool-reported timing/utilization can diverge from achieved frequency on a real board due to board-level constraints, DDR/IO, routing, thermal throttling, or different clocks. *Mitigation:* label all hardware throughput as estimated (not measured), and explicitly separate boardless evidence from on-board measurement in claims and limitations. *Residual risk:* absolute performance and power may differ materially on hardware.
- **T3 (High) — Partial Vivado coverage across benches.** The artifact explicitly records that B4/B5/B6 currently have `vivado_impl_ok=false`; therefore Vivado implementation evidence is not uniform across the benchmark suite. *Mitigation:* present Vivado results only where `vivado_impl_ok=true` (B1/B2/B3) and explicitly state failures/non-coverage for other benches. *Residual risk:* readers may overgeneralize if the paper's phrasing is not disciplined.
- **T4 (Medium) — Throughput is an estimate derived from reports, not a measured rate.** The table reports `hw_ticks_per_sec_est` with notes such as `estimated_from_clk_wns_ii` and marks `vivado=FAIL` where implementation did not succeed. This estimation can be sensitive to how `fmax_est` is derived and does not model runtime effects. *Mitigation:* interpret `hw_ticks_per_sec_est` only as a report-based estimate and always pair it with the CPU measured `cpp_ticks_per_sec` baseline as contextual anchor. *Residual risk:* estimates can mispredict by a large factor without on-board validation.

- **T5 (High) — Toolchain nondeterminism and version sensitivity.** HLS/Vivado are complex proprietary toolchains; QoR can change across versions, OS distributions, licensing settings, or machine environments. *Mitigation:* pin and report tool versions, record sha256 for all paper tables and key evidence, and treat QoR reproduction as same-toolchain-version evidence rather than a universal property. *Residual risk:* third-party environments may not reproduce identical QoR even when semantics/bit-exactness hold.
- **T6 (Medium) — Limited tick horizon on some benches.** Some verification runs use short tick counts (e.g., B4/B5 rows show `ticks=2`); short runs may miss long-horizon divergence modes (e.g., accumulation/saturation patterns) even if per-tick digests match for a few steps. *Mitigation:* keep the tick count explicit in the tables and position short-tick benches as workflow validation rather than maximal stress tests. *Residual risk:* bugs that appear only after many ticks remain unexercised for those benches.
- **T7 (Medium) — External-bundle dependency for B4.** The external connectome workflow depends on referenced files being present and having stable digests; missing or mutated external inputs can break reproducibility. *Mitigation:* require file existence and sha256 validation when provided, and treat external-verified status as part of the artifact contract (not as biological correctness). *Residual risk:* long-term availability and licensing/provenance of external files can remain outside the artifact's control.
- **T8 (Medium) — Bit-exactness depends on the serialization contract.** Correctness is mediated by a digest of the packed state vector; if the serialization/digest definition is wrong, digest equality could still hold while semantics drift. *Mitigation:* keep the bit-exactness definition explicit and narrow (digest over the specified representation), and pair it with trace presence and independent consistency checks where available. *Residual risk:* this is conformance to the stated contract, not a proof of equivalence of all internal/latent states.
- **T9 (High) — Audit circularity (self-certification).** A single auditing implementation (`audit_min`) could accidentally encode the same assumption/bug as the pipeline it validates, producing false confidence. *Mitigation:* the artifact includes an additional independent checker that runs in preflight and is reported separately (exit code and logs), reducing single-point-of-failure risk. *Residual risk:* the independent checker is still part of the same repository ecosystem and is not an external formal verifier.
- **T10 (Low) — Presentation/tooling drift (paper build warnings vs scientific results).** The evidence pack notes LaTeX warnings (e.g., undefined citations and an encoding anomaly). While this does not change the underlying JSON/CSV evidence, it can confuse reviewers or obscure traceability. *Mitigation:* treat paper-build cleanliness as an artifact-quality gate and fix encoding/citation issues proactively. *Residual risk:* none for correctness, but real risk for review reception and AE friction.

5.2 Limitations

- **No on-board measurement.** The artifact does not include measured power, measured wall-clock latency on a physical FPGA, or measured end-to-end throughput on a deployed bitstream; hardware evidence is boardless and report-derived.
- **Vivado implementation is not shown to succeed for all benches.** B4/B5/B6 are explicitly recorded as `vivado_impl_ok=false`; claims must not imply universal implementation success.
- **Throughput is reported as an estimate.** `hw_ticks_per_sec_est` is derived from report quantities (and is marked `vivado=FAIL` when applicable), so it is not a measured performance number.
- **Short-run verification for some workflows.** Rows with `ticks=2` validate workflow and contract adherence but do not establish long-horizon behavior under many-tick runs.
- **B5 is not a core, fully packaged benchmark in the current state.** The evaluation tables include a B5-family row, but it is not treated as a core correctness+implementation claim in the artifact story; treat it as non-core/experimental coverage.
- **Toolchain dependency.** Reproducing the hardware evidence requires access to the proprietary Xilinx/AMD toolchain (Vitis HLS / Vivado) and a compatible host environment.
- **Independent checking is separate, not formal.** The independent checker improves robustness against audit circularity but is not a formal proof system and is not external to the repository.
- **Paper-build hygiene is not yet perfect.** The evidence pack flags LaTeX issues (undefined citations, encoding anomaly) that should be fixed for a clean submission and for AE usability.

A Artifact Appendix

All commands are relative to the repository root.

1. **Regression suite (software sanity).**

   ```
   python -m pytest -q
   ```

   Expected: exit code 0. (Preflight logs are recorded under `build/paperA_preflight_out_v4/`.)
2. **Independent consistency check (anti-circularity support).**

   ```
   python tools/independent_check.py --paperA
   ```

Expected: exit code 0 (no independent consistency errors). Logs under `build/paperA_preflight_out_v4/`.

3. **Software gate (audit).**

   `python tools/audit_min.py --mode software`

   Expected: exit code 0 and JSON `decision=="GO"`.

4. **Bit-exactness verifier on core benchmarks (golden ↔ C++ reference).**

   ```
   python -m nema bench verify benches/B1_small/manifest.json \
   --outdir build/paper_verify/b1

   python -m nema bench verify benches/B3_kernel_302_7500/manifest.json \
   --outdir build/paper_verify/b3
   ```

   Expected (PASS) for each run: verifier JSON reports `ok==true` and `mismatches==[]` (i.e., mismatch=0).

5. **External bundle workflow (B4).**

   ```
   python -m nema bench verify benches/B4_real_connec
   --outdir build/paper_verify/b4
   ```

   Expected (PASS): verifier completes with `ok==true`. Contract expectation: referenced external files exist; if sha256 is not a placeholder token, it must match the file digest.

6. **Hardware pipeline generation (HLS + Vivado evidence capture).**

   `bash tools/run_hw_gates.sh`

   Expected (PASS by artifact contract): `build_hw/` contains at least one `**/bench_report.json` and report files under `**/hw_reports/**/*.(rpt|xml|log)` (or the equivalent set consumed by the report parser).

7. **Hardware gate (audit).**

   `python tools/audit_min.py --mode hardware`

   Expected (PASS): exit code 0 and JSON contains `decision=="GO"`; criteria include `hardwareToolchainAvailable=True`, `hardwareEvidenceG0b=True`, `hardwareEvidenceG2=True`, `hardwareEvidenceG3=True`, and `digestMatchAll=True`.

8. **Regenerate paper-facing tables/figures/evidence (refresh the pack).**

   `python tools/paperA/build_review_pack_v3.py`

   Expected: updates the paper tables (`results_bitexact.csv`, `results_qor.csv`, `results_throughput.csv`) and the coverage report (`vivado_coverage_report.md`) under `papers/paperA/artifacts/`, and refreshes the preflight outputs directory.

9. **Build the PDF.**

   `make -C papers/paperA clean paper`

   Expected: exit code 0; paper PDF produced at `papers/paperA/text/paper.pdf`.

10. **Sanity-check the pre-generated gate snapshot.**

    ```
    cat papers/paperA/artifacts/tables/gates_summary.md
    cat papers/paperA/artifacts/figures/gates_status.txt
    ```

    Expected: `gates_summary.md` shows both software and hardware with `decision=GO`; `gates_status.txt` exists.

11. **Optional (ergonomics; not a scientific claim): checkpoint/bundle regeneration.**

    `bash tools/checkpoint_full.sh`

    Expected: produces an evidence bundle and/or updated evidence directory consistent with the repository's checkpoint conventions.

A.1 Artifact Snapshot (pre-generated assets)

The appendix directly references the current artifact files in `papers/paperA/artifacts/`.

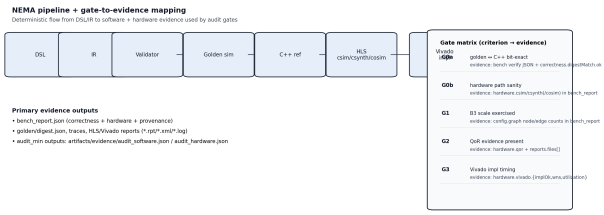

Figure 1. NEMA pipeline and gate-to-evidence mapping. G0a validates bit-exactness between golden and C++ reference (`bench verify` / `correctness.digestMatch.ok`); G0b validates hardware-path execution evidence (`csim/csynth/cosim` fields in `bench_report.json`); G1 validates that the 302/7500 benchmark evidence is present; G2 validates QoR/report evidence (`hardware.qor` and report files); G3 validates Vivado implementation timing/utilization evidence (`hardware.vivado.*`). Gate outputs and decisions are recorded in `papers/paperA/artifacts/evidence/audit_software.json` and `papers/paperA/artifacts/evidence/audit_hardware.json`.

Legacy status snapshot. `papers/paperA/artifacts/figures/gate` is retained as a lightweight artifact snapshot, but Figure 1 is the primary evidence-facing figure used by the paper.

References

[1] Willow Ahrens, James Demmel, and Hong Diep Nguyen. 2020. Algorithms for Efficient Reproducible Floating Point Summation. *ACM Trans. Math. Software* (2020). https://doi.org/10.1145/3389360

[2] Berk Atil, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, et al. 2024. Non-Determinism of "Deterministic" LLM Settings. https://doi.org/10.48550/arXiv.2408.04667 arXiv:2408.04667 [cs.CL]

[3] Heiko Becker, Robert Rabe, Eva Darulova, Magnus O. Myreen, Zachary Tatlock, Ramana Kumar, Yong Kiam Tan, and Anthony Fox. 2022. Verified Compilation and Optimization of Floating-Point Programs in CakeML. In *ECOOP 2022*. https://doi.org/10.4230/LIPIcs.ECOOP.2022.1

[4] Pierre Blanchard, Nicholas J. Higham, and Théo Mary. 2020. A Class of Fast and Accurate Summation Algorithms. *SIAM Journal on Scientific Computing* (2020). https://doi.org/10.1137/19M1257780

[5] Boyuan Chen, Mingzhi Wen, Yong Shi, Dayi Lin, Gopi Krishnan Rajbahadur, and Zhen Ming Jiang. 2022. Towards Training Reproducible Deep Learning Models. In *44th IEEE/ACM International Conference*

on Software Engineering (ICSE 2022). https://doi.org/10.1145/3510003.3510163

[6] Yu-Hsin Chou et al. 2020. Deterministic Atomic Buffering. In *53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-53)*. https://doi.org/10.1109/MICRO50266.2020.00083

[7] Travis DeWolf, Pawel Jaworski, and Chris Eliasmith. 2020. Nengo and low-power AI hardware for robust, embedded neurorobotics. https://doi.org/10.48550/arXiv.2007.10227 arXiv:2007.10227 [cs.RO]

[8] Mingzhe Gao, Jieru Zhao, Zhe Lin, and Minyi Guo. 2024. Hierarchical Source-to-Post-Route QoR Prediction in High-Level Synthesis with GNNs. https://doi.org/10.48550/arXiv.2401.08696 arXiv:2401.08696 [cs.AR]

[9] Pingakshya Goswami, Benjamin Carrion Schaefer, and Dinesh Bhatia. 2023. Machine learning based fast and accurate High Level Synthesis design space exploration: From graph to synthesis. *Integration* (2023). https://doi.org/10.1016/j.vlsi.2022.09.006

[10] Yann Herklotz, James D. Pollard, Nadesh Ramanathan, and John Wickerson. 2021. Formal Verification of High-Level Synthesis. *Proceedings of the ACM on Programming Languages* (2021). https://doi.org/10.1145/3485494

[11] Yann Herklotz and John Wickerson. 2024. Hyperblock Scheduling for Verified High-Level Synthesis. *Proceedings of the ACM on Programming Languages* (2024). https://doi.org/10.1145/3656455

[12] Mehrzad Karamimanesh, Ebrahim Abiri, Mahyar Shahsavari, Kourosh Hassanli, André van Schaik, and Jason Eshraghian. 2025. Spiking neural networks on FPGA: A survey of methodologies and recent advancements. *Neural Networks* (2025). https://doi.org/10.1016/j.neunet.2025.107256

[13] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* (2009). https://doi.org/10.1145/1538788.1538814

[14] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. *Proceedings of the ACM on Programming Languages* (2021). https://doi.org/10.1145/3453483.3454030

[15] Rachit Nigam, Samuel Thomas, Zhijing Li, and Adrian Sampson. 2021. A Compiler Infrastructure for Accelerator Generators. In *ASPLOS '21*. https://doi.org/10.1145/3445814.3446712

[16] Jens E. Pedersen et al. 2023. Neuromorphic Intermediate Representation: A Unified Instruction Set for Interoperable Brain-Inspired Computing. https://doi.org/10.48550/arXiv.2311.14641 arXiv:2311.14641 [cs.NE]

[17] Jens E. Pedersen et al. 2024. Neuromorphic intermediate representation: A unified instruction set for interoperable brain-inspired computing. *Nature Communications* (2024). https://doi.org/10.1038/s41467-024-52259-9

[18] Hung Viet Pham, Shangshu Qian, Jiannan Wang, Thibaud Lutellier, Jonathan Rosenthal, et al. 2020. Problems and Opportunities in Training Deep Learning Software Systems: An Analysis of Variance. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20)*. https://doi.org/10.1145/3324884.3416545

[19] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS '98)*. https://doi.org/10.1007/BFb0054170

[20] Louis-Noël Pouchet, Emily Tucker, Niansong Zhang, Hongzheng Chen, Debjit Pal, Gabriel Rodríguez, and Zhiru Zhang. 2024. Formal Verification of Source-to-Source Transformations for HLS. In *FPGA '24*. https://doi.org/10.1145/3626202.3637563

[21] Xiaojia Rao, S. Radziuk, Conrad Watt, and Philippa Gardner. 2025. Progressful Interpreters for Efficient WebAssembly Mechanisation. *Proceedings of the ACM on Programming Languages* (2025). https://doi.org/10.1145/3704858

[22] Aysha Ravi et al. 2025. Improving the Reproducibility of Deep Learning Software: An Initial Investigation through a Case Study Analysis. https://doi.org/10.48550/arXiv.2505.03165 arXiv:2505.03165 [cs.SE]

[23] Shanmugavelu et al. 2024. Impacts of floating-point non-associativity on reproducibility for HPC and deep learning applications. https://doi.org/10.48550/arXiv.2408.05148 arXiv:2408.05148 [cs.DC]

[24] Megha Srivastava, Simran Arora, and Dan Boneh. 2024. Optimistic Verifiable Training by Controlling Hardware Nondeterminism. https://doi.org/10.48550/arXiv.2403.09603 arXiv:2403.09603 [cs.CR]

[25] Zi Wang and Benjamin Carrion Schafer. 2022. Learning from the Past: Efficient High-level Synthesis Design Space Exploration for FPGAs. *ACM Transactions on Design Automation of Electronic Systems* (2022). https://doi.org/10.1145/3495531

[26] Conrad Watt, Xiaojia Rao, Jean Pichon-Pharabod, Martin Bodin, and Philippa Gardner. 2021. Two Mechanisations of WebAssembly 1.0. In *Formal Methods (FM 2021)*. https://doi.org/10.1007/978-3-030-90870-6_4

[27] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *PLDI '11*. https://doi.org/10.1145/1993316.1993532

[28] Hanchen Ye et al. 2022. ScaleHLS: a Scalable High-Level Synthesis Framework with Multi-level Transformations and Optimizations. In *DAC '22*. https://doi.org/10.1145/3489517.3530631

[29] Hanchen Ye, Cong Hao, Jianyi Cheng, Hyunmin Jeong, Jack Huang, Stephen Neuendorffer, and Deming Chen. 2021. ScaleHLS: A New Scalable High-Level Synthesis Framework on Multi-Level Intermediate Representation. https://doi.org/10.48550/arXiv.2107.11673 arXiv:2107.11673 [cs.AR]

[30] D. Youn et al. 2024. Bringing the WebAssembly Standard up to Speed with SpecTec. *Proceedings of the ACM on Programming Languages* (2024). https://doi.org/10.1145/3656440

[31] Donglin Zhuang, Xingyao Zhang, Shuaiwen Leon Song, and Sara Hooker. 2021. Randomness In Neural Network Training: Characterizing The Impact of Tooling. https://doi.org/10.48550/arXiv.2106.11872 arXiv:2106.11872 [cs.LG]